

Good Cache, BAD Cache: Exploiting DNSSEC Validation Failures for DNS Cache Poisoning Attacks

Shiming Liu*
liusm23@mails.tsinghua.edu.cn
Tsinghua University, BNRist
Beijing, China

Yunyi Zhang*
yunyizhang@mail.tsinghua.edu.cn
Tsinghua University
Beijing, China

Chaoyi Lu
lucy@zgclab.edu.cn
Zhongguancun Laboratory
Beijing, China

Baojun Liu†
lbj@tsinghua.edu.cn
Tsinghua University
Beijing, China

Shuhan Zhang
zhangsh22@mails.tsinghua.edu.cn
Tsinghua University
Beijing, China

Donghong Sun
sundonghong@tsinghua.edu.cn
Tsinghua University
Beijing, China
Zhongguancun Laboratory
Beijing, China

Yong Ma
mayong.my@alibaba-inc.com
Alibaba Group
Beijing, China

Linjian Song
linjian.slj@alibaba-inc.com
Alibaba Group
Beijing, China

Abstract

Cache poisoning has been a persistent adversarial challenge in the DNS for decades. DNSSEC (Domain Name System Security Extensions) verifies DNS resource records by digital signatures and is widely regarded as the best practice for mitigating cache poisoning attacks.

In this paper, we present BADDNS, a novel DNS cache poisoning attack, which exploits the improper handling of a new type of cache in the resolver to undermine DNSSEC validation. Specifically, when DNSSEC validation for a domain fails, some resolvers still cache the resource records from the response, storing them in what is known as a “BAD cache.” Then, these resolvers incorrectly use these unverified resource records in subsequent resolution processes, creating a potential attack vector. By systematically analyzing and reviewing the BAD cache implementations in six major software packages, we confirmed that BADDNS is practically exploitable, affecting three of these implementations, including PowerDNS and Microsoft DNS. With comprehensive measurements, we discovered that approximately 16.86% of the 300,792 DNSSEC-validating open resolvers and 10 public DNS providers are susceptible, including OpenDNS and Quad9. Moreover, we found that 26,930 DNSSEC-enabled nameservers are vulnerable, including Cloudflare and INWX GmbH, which affects 5.68% of the domains they serve. We responsibly disclosed our findings to the affected vendors, along with detailed mitigation advice. We have since received positive

responses from 7 vendors, such as PowerDNS, several of whom are now implementing our proposed solutions.

CCS Concepts

• **Networks** → **Naming and addressing**; • **Security and privacy** → **Network security**.

Keywords

Domain name system, DNSSEC, Cache poisoning attack, BAD cache

ACM Reference Format:

Shiming Liu, Yunyi Zhang, Chaoyi Lu, Baojun Liu, Shuhan Zhang, Donghong Sun, Yong Ma, and Linjian Song. 2026. Good Cache, BAD Cache: Exploiting DNSSEC Validation Failures for DNS Cache Poisoning Attacks. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3830454.3832699>

1 Introduction

The Domain Name System (DNS) is the basis of almost every Internet communication. Cache poisoning is one of the most severe threats to the DNS. Attackers can hijack user traffic by injecting forged resource records into DNS resolvers. To counter it, the security community introduced DNSSEC (Domain Name System Security Extensions) [53], which uses digital signatures to verify the validity of resource records. DNSSEC is widely regarded as the best practice for defending against cache poisoning. Although numerous cache poisoning attacks have emerged in recent years, including birthday attacks [37, 57], IP fragmentation-based techniques [25, 69], ICMP-based side channels [38, 39], DNS response pre-processing flaws [36], and bailiwick vulnerabilities [35, 65], these attacks target resolvers or domains that do not enforce DNSSEC, and thus cannot compromise the protection offered by DNSSEC. Prior measurements [4, 12] further suggest that the signing rate of the top-level domains (TLDs) exceeds 90%,

*Both authors contributed equally to this research.

†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *CCS '26, The Hague, Netherlands*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2871-6/2026/11

<https://doi.org/10.1145/3830454.3832699>

and more than 40% of open resolvers in the wild perform DNSSEC validation.

However, DNSSEC increases the complexity of the DNS ecosystem, including various signature algorithms, new resource records (e.g., DNSKEY, RRSIG), and an additional validation process. This complexity may also introduce new potential risks. Zhang et al. [66] leveraged DNSSEC debugging mechanisms to mount denial-of-service attacks, while Heftrig et al. [24] exploited resolver incompatibilities with new DNSSEC algorithms to force resolvers to downgrade DNSSEC. Distinct from prior work, our study investigates the long-overlooked impact of DNSSEC on unsigned domains in a properly deployed DNSSEC environment, revealing that under certain conditions, *DNSSEC can transform from a defender against cache poisoning into an accomplice*. We demonstrate that *DNSSEC-validating resolvers can be induced to cache and reuse forged resource records of domains protected by DNSSEC*, hijacking the unsigned domains delegated to them.

BAD cache in DNSSEC. Our attack exploits a special type of caching inherent in DNSSEC. The BAD cache, a specialized cache introduced by DNSSEC and defined in RFC 4035 [54], is intended for the temporary storage of resource records that fail DNSSEC validation. Its purpose is similar to that of a negative cache—to reduce futile resolution attempts. However, unlike a negative cache, the BAD cache is optional and is meant to be used only within the DNSSEC validation logic. Furthermore, RFC 4035 specifies additional constraints: (i) a resolver should set a small TTL for “BAD cache” entries, rather than trusting the TTL from the original response, and (ii) “BAD cache” entries should only be used to short-circuit the DNSSEC validation process.

Our study. To investigate the potential risks that resolvers face in specific caching scenarios, we conducted a systematic review of the BAD cache definition in DNS RFCs and analyzed 6 mainstream DNS software packages that support DNSSEC, including BIND9, Unbound, Knot, and PowerDNS. By inspecting their source code and conducting black-box testing, we found significant differences in their implementations of the BAD cache, including *what is stored, for how long it is stored, and how the stored resource records are used*. We confirmed that some resolvers conflate the usage boundaries of the BAD cache. Specifically, when a response from a nameserver fails DNSSEC validation, these resolvers still cache the resource records from that response. These validation-failed resource records are then used in subsequent resolution processes for both DNSSEC-enabled and non-DNSSEC-enabled queries alike. Leveraging these observations, we introduce a novel off-path cache poisoning attack, which we name BADDNS. Different from the previous off-path cache poisoning attacks like TuDoor [36] or SADDNS [38, 39] that focus on DNSSEC-disabled resolvers, BADDNS exploits a cache misuse vulnerability in DNSSEC-enabled environments. For a domain with a proper DNSSEC deployment (e.g., *vitm-ns.com*), an attacker can craft a response designed to fail validation, thereby injecting a forged resource record (e.g., `<dns.vitm-ns.com A a.t.k.r>`) into the resolver’s BAD cache. Then, when a resolver receives a query for another domain that is delegated to the compromised nameserver, even though the nameserver correctly deploys DNSSEC, the resolver skips DNSSEC validation and directly uses the attacker-injected forged resource records from the BAD cache to complete subsequent resolution steps.

We demonstrated that BADDNS is a practical and widespread threat. First, the scenario where an authoritative nameserver for a non-DNSSEC-enabled domain is itself DNSSEC-enabled is common. Our analysis of the Tranco Top 1M domains shows that 35.88% of domains meet this condition. We further tested their vulnerability and found that among the 26,930 nameservers these domains rely on, 9,055 (33.62%) are vulnerable, including Cloudflare, placing 5.68% of the domains at risk of the BADDNS attack, such as Shopify [56] and APKPure [3]. Second, we performed an end-to-end poisoning demonstration against a widely deployed resolver and conducted a large-scale impact evaluation on DNSSEC-supporting public DNS services and stable open resolvers. The results show that 16.86% of open resolvers and 10 major public DNS providers are vulnerable to BADDNS, like OpenDNS [44], Quad9 [52], and UltraDNS [61]. **Mitigation and disclosure.** Our findings highlight a gap between protocol design and implementation: iterative updates to the protocol can introduce unforeseen complexities to key features, which in turn can undermine the security logic of existing implementations. To address this, we first call for protocol designers to provide clear, self-contained specifications for extensions to critical features, similar to RFC 2308 [2] for negative caching, to avoid ambiguity in security semantics. For specific implementations, we recommend that resolvers should only cache a marker (e.g., the `<QNAME, QTYPE, QCLASS>` tuple) for validation failures, not the full records. Furthermore, a uniform, small TTL should be enforced for all BAD cache entries, and this cache must be strictly segregated from other caches to limit its scope of use. With respect to DNSSEC, our work highlights potential issues in current deployment practices and serves as a call for all stakeholders in the DNS ecosystem to jointly advance its complete and effective implementation.

Finally, we have responsibly disclosed our results to affected vendors and shared concrete hardening guidance. We have received acknowledgments from PowerDNS and Knot, and are working in close collaboration with them to develop mitigation strategies. We are still waiting for responses from other vendors.

Contributions. We make the following contributions:

Systematic analysis of BAD cache implementations. We conduct a source code audit and black-box experiments on six major DNS software packages to analyze their caching behavior following a DNSSEC validation failure and to summarize the potential weaknesses of resolver caching operations in DNSSEC scenarios.

Novel attack. We propose a novel off-path attack BADDNS that achieves cache poisoning even when DNSSEC is properly deployed, a significant advance in the study of DNSSEC threats.

Comprehensive evaluation. We perform end-to-end experiments on major resolver implementations and conduct large-scale measurements of public DNS services, open resolvers, and popular domains, confirming the widespread impact of BADDNS.

Responsible disclosure. We have since responsibly disclosed our findings to all affected service providers and have proposed practical mitigation measures.

2 Background and Related Works

2.1 DNSSEC

DNSSEC enhances DNS security by adding encrypted signatures to existing DNS records [54, 55]. These signatures allow verification

of the requested DNS records to ensure that their origin is from the genuine authoritative nameserver.

How DNSSEC works. DNSSEC augments DNS with public-key signatures over RRsets so that a validating resolver can verify origin authenticity and data integrity [54, 55]. Each signed RRset is accompanied by an RRSIG created with the zone’s private key; the resolver retrieves the corresponding DNSKEY and verifies the signature. Trust is bootstrapped from a locally configured root trust anchor and propagated down the hierarchy via DS → DNSKEY: a parent publishes a signed DS for the child, the child publishes (and signs) its DNSKEY, and that key signs the child’s data. During the process of validation, if every link validates and the target records’ RRSIGs pass DNSSEC validation, the answer is proven as secure. If a needed link is provably wrong (e.g., DS/DNSKEY mismatch), or RRSIGs fail DNSSEC validation, the answer is considered as *bogus* and returns SERVFAIL [54].

In the wild, DNSSEC deployment is uneven: Only a small subset of second-level domain (SLD) zones are signed, while many dependent delegations remain unsigned or misconfigured [11, 12]. For a validating resolver, this mixed reality is handled by policy in the tree walk. Concretely, when descending from a signed parent into a child delegation that has no DS at the parent, the resolver must treat the child as *Insecure* and stop requiring DNSSEC validation for that branch [54].

DNSSEC records. In order to implement the signature verification, several new types of RRs are introduced in DNSSEC, including RRSIG and NSEC/NSEC3.

- **RRSIG.** Every signed RRset carries an RRSIG that encodes the signing algorithm, the validity window, the key tag, and the cryptographic signature [55]. These signatures add nontrivial payload—especially with RSA or when multiple algorithms/keys are present—and therefore materially increase response sizes [27].

- **NSEC/NSEC3.** To confirm either that the queried name does not exist (NXDOMAIN) or the RR type at an existing name does not exist [55], resolvers expect signed NSEC or NSEC3 proofs. Typical NXDOMAIN replies from DNSSEC-enabled authorities include the zone’s SOA+RRSIG and one or more NSEC/NSEC3+RRSIG in the AUTHORITY section [6]. For wildcard synthesis, answers contain the synthesized RRset (e.g., A+RRSIG) with proofs that the wildcard applies and no closer name blocks it [6].

2.2 BAD Cache

DNSSEC introduces additional operational complexity, including the possibility of validation failures due to administrative misconfigurations (e.g., expired signatures, missing keys, or timing mismatches) [18]. Such failures are often persistent and not resolved by repeated queries. Without mitigation, these persistent errors can lead to excessive and redundant DNS traffic, burdening both resolvers and authoritative servers [54].

To address this, RFC 4035 introduces the “BAD cache” — a mechanism by which validating resolvers temporarily cache RRsets that failed DNSSEC validation [54]. Unlike traditional positive or negative caches that store trustworthy records or valid non-existence information, the BAD cache serves a defensive role: it aims to suppress repeated queries for data that is known to be invalid due to

signature verification failures, thereby reducing load on the DNS infrastructure without compromising security [54].

Figure 1 illustrates the caching behaviors of a DNSSEC-validating recursive resolver when handling responses with valid and invalid signatures. We use *valid.com* and *invalid.com* as representative examples. Both domains are DNSSEC-signed; however, due to differences in authoritative server configurations, *valid.com* successfully provides the correct cryptographic material for validation, while *invalid.com* either fails to provide the necessary DNSSEC records or returns incorrect ones. When a client queries *valid.com*, the resolver retrieves an A record along with a valid RRSIG, verifies the signature, and caches the record in the positive cache. The client receives a NOERROR response, and future queries can be answered directly from the cache until the TTL expires. In contrast, when querying *invalid.com*, the resolver receives an A record with an invalid RRSIG. The DNSSEC validation fails, and the response is not inserted into the positive cache. Instead, a BAD cache entry is created to record the RRsets that failed validation, and the client receives a SERVFAIL. For the same future queries, the resolver avoids the validation and returns SERVFAIL directly until the TTL expires.

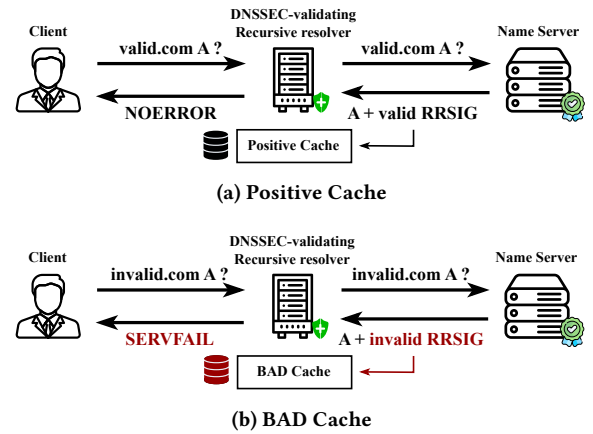


Figure 1: Caching behavior of DNSSEC-validating resolvers for valid and invalid DNSSEC signatures.

While both positive cache and BAD cache aim to improve resolver efficiency, they differ significantly in their functions and usages.

Data trustworthiness and purpose. Positive cache stores RRsets that have passed validation (or do not require validation in non-DNSSEC scenarios), and these cached entries can be returned directly to clients. In contrast, BAD cache stores information about failed validation attempts. The core assumption is that these data are not trustworthy, and therefore must not be reused in standard resolution with DNSSEC unless explicitly allowed under specific circumstances (e.g., clients querying with CD bit) [54].

TTL behavior. For positive cache, the TTL is taken directly from the RRset received from the authoritative server, as specified by the zone operator. This TTL governs how long the record can be served from cache. However, for BAD cache, the response is not considered trustworthy, so the TTL from the original response must

not be used. Instead, the resolver implementation should assign a custom, small TTL (e.g., 10 seconds) to limit the duration of the BAD cache entry [54].

Summary. The implementation of BAD cache requires strictly following certain behavioral constraints. First, they must not return RRsets from the BAD cache unless validation is explicitly disabled. What's more, they must assign a short, locally defined TTL to BAD cache entries instead of the original, potentially untrustworthy record TTL [54]. However, the implementation of BAD cache is optional [63], which leads to observable differences across resolver implementations, not only in whether BAD cache is implemented, but also in how its behavior and functionality are realized.

2.3 DNS Cache Poisoning Attacks

DNS cache poisoning refers to injecting forged or manipulated resource records into a resolver's cache so that subsequent clients are misdirected to attacker-chosen endpoints. The consequences include traffic hijacking and phishing. Over the past two decades, attackers have developed multiple families of cache-poisoning techniques; below we summarize three predominant lines: *forgery-based*, *fragmentation-based*, and *DNSSEC-downgrade* attacks.

Forgery-based attacks. The goal in forging attacks is to craft a rogue DNS response and have the resolver accept it before the genuine reply arrives. Concretely, acceptance requires matching the question section, transaction ID (TXID), and the 4-tuple $\langle \text{srcIP}, \text{dstIP}, \text{srcPort}, \text{dstPort} \rangle$. If an attacker races a spoofed response that satisfies these fields, the resolver may cache the attacker's records.

The most influential instance is the 2008 Kaminsky attack [30], which exploited predictable query flows to amplify the probability of a correct guess. To counter this class, RFC5452 mandates strong randomization of ephemeral source ports and TXIDs[26], making naive brute force (over a nominal 2^{32} space) impractical.

Later work demonstrated how vulnerabilities can erode this entropy. SADDNS [38, 39] leverages side channels to “divide and conquer,” first inferring the source port and then the TXID, reducing the effort to roughly $2^{16} + 2^{16}$ spoofed replies and extending the effective attack window. TuDoor [36] exploits resolver-preprocessing weaknesses where some implementations on specific platforms use sequential or otherwise predictable ports/TXIDs, enabling poisoning against targets.

Fragmentation-based attacks. A different line of work bypasses query/port randomization by abusing IP reassembly. The second fragment of a UDP/DNS packet contains neither UDP headers nor the question, so randomization defenses guarding those fields do not apply to it.

Prior studies [9, 25, 69] show that an attacker can (i) pre-send a spoofed *second* fragment carrying malicious RRs, (ii) trigger the resolver to elicit a large response from an authority, and (iii) rely on the resolver's IP reassembly to combine the legitimate first fragment with the spoofed second fragment into a single, cacheable response. Two main strategies increase the chance of fragmentation: lowering the path MTU (e.g., via PTB signals) and soliciting oversized answers (e.g., DNSSEC proofs). Subsequent work also built large responses with CNAME chains so that fragmentation happens at the resolver and the reassembled message poisons a downstream forwarder [69].

These techniques avoid guessing TXID, but earlier variants often assumed very small fragment thresholds or targeted forwarders rather than full recursive resolvers, limiting applicability.

DNSSEC-downgrade attacks. A third family exploits cryptographic agility in DNSSEC. When resolvers encounter responses listing only unsupported algorithms, some implementations have been shown to skip validation, effectively treating data as unsigned. Under on-path control, an adversary can inject such algorithm manipulations to force validation to be bypassed and then combine this with a traditional forgery to poison caches [24]. These attacks target resolvers with lax handling of algorithm negotiation and, crucially, require on-path capabilities.

Limitations of previous attacks. While the above attacks have been shown to achieve cache poisoning in mainstream resolver implementations, their threat models largely focus on resolvers that are *not* protected by DNSSEC. In other words, once a resolver enforces DNSSEC validation for the target domain, the forged data is rejected, and the above attack can not succeed. For DNSSEC-downgrade attacks, the adversary typically must be *on-path* to do manipulation, which substantially narrows the attacker model.

In this paper, we focus on whether an *off-path* adversary can still induce cache poisoning *against DNSSEC-validating resolvers* by leveraging the interaction with BAD cache, thereby demonstrating potential risks of DNSSEC and calling on the community to advance and strengthen DNSSEC adoption.

3 Software Implementations of BAD Cache

This section investigates the implementation and behavior of BAD cache mechanisms across mainstream DNS resolvers that support DNSSEC validation. Our investigation focuses on whether these resolvers implement BAD cache, what content gets stored, how long it is retained, and whether it is placed in a separate BAD cache or conflated with the positive cache.

We begin by conducting black-box tests, sending resolvers carefully crafted responses that fail DNSSEC validation, and observing their subsequent behavior. Then, we perform a white-box analysis by inspecting the source code and using GDB to trace the runtime behavior of these implementations. Section 3.1 and Section 3.2 describe our experimental setup and methodology, and Section 3.3 presents our findings on BAD cache analysis.

3.1 Experimental Setup

To investigate the practical implementations of BAD cache in mainstream DNSSEC-validating resolvers, we design a controlled experimental environment deployed entirely within a closed LAN using virtual machines. The testbed consists of three core components: a client acting as the DNS query initiator and response logger, a target resolver instance under evaluation, and a customized DNS infrastructure composed of multiple authoritative servers. All machines in the test environment run Ubuntu 22.04 LTS, except for tests involving Microsoft DNS Server, which is deployed on a Windows Server 2022 Datacenter with the latest patches applied.

The authoritative servers are powered by BIND 9.21.0 and configured to host two zones: `nameserver.com` and `user.com`. The `nameserver.com` zone is fully DNSSEC-signed and simulates a DNS hosting provider through the subdomain `dns.nameserver.com`.

Table 1: Implementation behavior of BAD cache across 6 mainstream DNSSEC-validating resolvers.

Resolver	Version	Validation-failed RRSet Cached	Max TTL	Cache Entry ¹	Direct Reuse ²	Indirect Reuse (DNSSEC) ³	Indirect Reuse (non-DNSSEC) ⁴
Knot Resolver [17]	6.0.14	✓	600	Type I	✗	✓	✓
PowerDNS Recursor [48]	5.2.2	✓	3,600	Type I	✗	✗	✓
Microsoft DNS [40]	2022	✓	180	Type I	✗	✓	✓
Unbound [43]	1.23.0	✓	60	Type I	✗	✗	✗
BIND9 [28]	9.21.9	✗	-	Type II	-	-	-
Technitium [59]	13.6	✗	-	Type III	-	-	-

¹ Type I: whole RRset; Type II: <QNAME, QTYPE>; Type III: failed DNSSEC material.

² Whether a cached failed RRset is returned directly in response to a repeated query.

³ Whether the failed RRset is reused internally during the resolution of other DNSSEC-validated names.

⁴ Whether the failed RRset is reused during the resolution of other names in a non-DNSSEC context.

'-': Not Applicable for resolvers that do not cache validation-failed RRsets. ✓: Yes. ✗: No.

Notably, the RRSIG record for `dns.nameserver.com` can be deliberately corrupted to ensure consistent DNSSEC validation failures. The `user.com` zone, representing an end-user domain, is delegated to `dns.nameserver.com`. The experimental results in Section 6.3 show that this configuration is common in the wild due to the challenges of DNSSEC deployment. To support flexible testing across multiple subdomains, the `user.com` zone configures a wildcard record (i.e., `*.user.com`), enabling on-demand creation of arbitrary subdomains during evaluation.

3.2 Methodology

Our evaluation targets include widely used, actively maintained DNS resolver implementations with DNSSEC validation support. All tested software is drawn from their latest stable releases, either from official package repositories or directly from vendor distributions. The complete list of tested resolvers is presented in Table 1, which is derived from prior works [36, 67].

To understand the caching behavior following DNSSEC validation failures, we employed both black-box and white-box analysis techniques.

Black-box tests. The primary goal of the black-box phase is to determine whether a resolver caches a response that fails DNSSEC validation, and if so, what information is retained, for how long, and how it is subsequently utilized in the resolution process. To populate a cache entry with invalid DNSSEC data, the client first queries the A record of `dns.nameserver.com` with the DO flag set, ensuring DNSSEC validation is explicitly triggered and fails. Subsequent queries are issued to test different caching scenarios:

- **Used directly.** The client reissues the query for `dns.nameserver.com`, again with the DO flag. This evaluates whether the failed RRset is served directly from the cache or revalidated from scratch.

- **Used in process with DNSSEC context.** The client then queries a randomly generated subdomain under `user.com`, such as `abc123.user.com`, which is resolved through a wildcard record. In this scenario, the zone is DNSSEC-signed, and the goal is to determine whether the resolver reuses the `dns.nameserver.com` in BAD cache during future resolution. Using random subdomains guarantees that no valid cache entry from earlier queries is reused, thereby isolating the contribution of BAD cache, if present.

- **Used in process without DNSSEC context.** The client queries a randomly generated subdomain under `user.com` without enabling DNSSEC for the zone. This case allows us to examine whether resolvers bypass BAD cache restrictions when the validation context is not enforced, and whether the cached invalid data is still utilized as part of the resolution chain.

To determine the lifespan of BAD cache entries, we implemented an active probing routine in which the client repeatedly issues queries every second. This process continues until the resolver does not respond directly within a small time but initiates a new query to the target nameserver, allowing us to infer the effective TTL assigned to the BAD cache entry.

White-box tests. While black-box methods provide surface-level insight, they were insufficient in cases where observed behavior contradicts documentation or lacks clarity. Therefore, we complemented our analysis with white-box techniques. For resolvers exhibiting ambiguous behavior—those claiming to implement BAD cache yet diverging from RFC expectations—we engaged in source code inspection, dynamic tracing using GDB, runtime logging analysis, and direct introspection of internal cache states, where supported. In addition, for resolvers that did not exhibit observable BAD cache characteristics under our black-box tests, we applied the same white-box workflow to locate their concrete caching logic.

3.3 Results

Table 1 summarizes resolver behaviors in caching and reusing responses under DNSSEC validation failure. Our experiments reveal notable divergences in how DNS resolvers implement and utilize BAD cache. These differences manifest in whether resolvers cache validation-failed records at all, how long such records persist, and whether they are then reused across resolution tasks.

Overall, three of six tested resolvers cache full RRsets when DNSSEC validation fails, including Knot Resolver, PowerDNS Recursor, and Microsoft DNS. Their handling follows a common pattern: records that fail validation are explicitly tagged with a failure status (e.g., “bogus” or “bad”) and then admitted into cache. Crucially, the BAD cache is subsequently reused by the resolver’s ordinary resolution pipeline. Their TTL enforcement varies widely: some impose strict limits (e.g., Unbound for 60s and Microsoft DNS for 180s), while others, such as PowerDNS Recursor, retain a 3,600s-long TTL. Furthermore, the extent to which resolvers reuse the

BAD cache—either as direct answers or during resolution delegation—varies from these tested resolvers. Below, we analyze each resolver’s behavior in detail.

- **Knot Resolver.** Knot Resolver caches validation-failed RRsets with a default TTL of 600 seconds and permits their reuse during indirect resolution. When a domain such as `dns.nameserver.com` fails validation, Knot tags the record accordingly [33] and prevents direct reuse. However, in both DNSSEC-validating and non-validating contexts, the cached entry [32] is still leveraged during the resolution of dependent names (e.g., `www.user.com`), allowing BAD data to propagate across the resolution process [31].

- **PowerDNS Recursor.** PowerDNS Recursor follows a similar caching strategy. After encountering a DNSSEC validation failure, it caches the RRset [50] and annotates it with a “Bogus” label [51]. These records are retained for up to 3,600 seconds. Unlike Knot Resolver, PowerDNS Recursor avoids reuse in DNSSEC-validating scenarios but does allow reuse when DNSSEC is not enabled [49].

- **Microsoft DNS.** Microsoft DNS caches validation-failed RRsets but does not return them directly to clients. Instead, it only consults these entries during iterative processing for other names—both in DNSSEC-validating and non-validating contexts—thereby enabling indirect reuse during delegation. Crucially, Microsoft DNS overrides upstream TTLs and caps such entries to 180 seconds.

- **Unbound.** Unbound implements a stricter cache mechanism. While it does cache failed RRsets temporarily (for up to 60 seconds), it explicitly prohibits their reuse in all contexts [62]. Cached entries are labeled as bogus, and any subsequent resolution path that encounters such a record triggers a fresh upstream query. This approach avoids any possibility of invalid data.

- **BIND9.** BIND9 does not store the full RRset following a DNSSEC validation failure. Instead, it only records a `<QNAME, QTYPE>` tuple in its internal cache with a short-lived timer [8]. The failed record is neither reused as a direct response nor during intermediate resolution stages. This approach minimizes the risk of invalid data.

- **Technitium.** Technitium DNS also avoids storing RRsets that fail DNSSEC validation. Although its BAD cache retains cryptographic metadata such as DNSKEY or DS when validation fails, it discards A or AAAA records entirely [58]. This ensures that invalid address records are never reused, even indirectly. The resolver instead issues new upstream queries for each request, regardless of previous failures.

3.4 Summary

Our implementation analysis reveals BAD cache misuse across multiple mainstream resolvers. Knot Resolver, PowerDNS Recursor, and Microsoft DNS all retain and later use RRsets that failed DNSSEC validation during iterative processing in future resolution. This means that, despite a domain being properly secured with DNSSEC, the resolver still trusts and uses the attacker-injected forged resource records, thereby undermining the security guarantees provided by DNSSEC. Although RFC 4035 permits maintaining a “BAD cache” with strict safeguards (e.g., short TTLs and no direct answers), the observed behaviors cross the security boundary that DNSSEC intends: they allow validation-failed data to influence

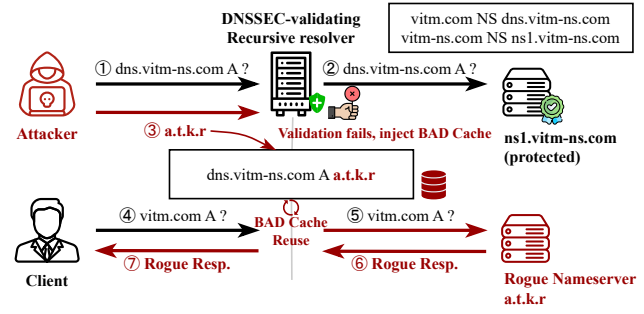


Figure 2: Threat model of BADDNS attack.

nameserver selection for otherwise ordinary lookups. Compounding this, TTL handling is inconsistent—e.g., 600s (Knot), 3,600s (PowerDNS), and 180s (Microsoft)—and BAD entries are often insufficiently segregated from normal resolution logic. These behaviours lead to an exploitable policy gap: once an attacker can inject bogus information as *BAD*, resolvers will still reuse it in delegation, enabling traffic steering to an attacker-controlled nameserver and subsequent insertion of forged resource records into the positive cache. These discrepancies explain why, in later sections, we can escalate from a transient validation failure to persistent cache poisoning, despite resolvers “following the RFC” in the narrow sense of not answering directly from the BAD cache.

4 BADDNS Attack

Building on the study of mainstream resolver implementation in Section 3.3, we demonstrated that some software implementations cache resource records from responses when DNSSEC validation fails, and subsequently use this data in future resolution processes. This behavior, which conflates the usage boundaries of the BAD cache, introduces potential security risks. In this section, we introduce the threat model that exploits the BAD cache to poison the cache of recursive resolvers, then present the attack workflow.

4.1 Threat Model

Figure 2 illustrates our threat model. The attacker’s objective is to hijack the target domain by poisoning the resource records of its authoritative nameserver in a well-deployed DNSSEC environment, where resolvers enforce DNSSEC validation, and the authoritative nameserver zone is properly protected by DNSSEC. In this threat model, although the target domain’s authoritative zone is correctly configured with DNSSEC, the attacker can still compel the recursive resolver to use forged resource records, thereby influencing subsequent resolution processes (③–⑥).

To carry out this attack, the following two conditions must be satisfied. (i) The target authoritative nameserver is properly configured with DNSSEC and is not vulnerable to known attacks like DNSSEC downgrades [24], while there are unsigned domains hosted on that nameserver. This scenario is quite common, given the currently high deployment rate of DNSSEC. It is particularly prevalent in DNS hosting services (e.g., Cloudflare), where the provider’s nameservers are often DNSSEC-enabled, even though the customer domains they host are not. (ii) The target resolver should be DNSSEC-enabled

and implement BAD cache. Our experimental results in Section 6.2 have confirmed that 16.86% of DNSSEC-validating open resolvers in the wild meeting this condition.

The adversary is off-path and does not break DNSSEC cryptography. Instead, it can spoof source IP addresses and run an authoritative nameserver under its control. These assumptions are realistic: recent Internet measurements report that more than one-fifth of IPv4 autonomous systems still permit source address spoofing [10], making off-path packet injection feasible at scale. Likewise, operating an authoritative DNS service is trivial—even no domain registration is required; the attacker can simply rent a commodity cloud VM and publicly provide an authoritative name service as a rogue nameserver.

4.2 Attack Workflow

In this section, we first provide an overview of the BADDNS workflow as shown in Figure 3, and then present the concrete attack details: fragmentation, the triggering of large responses, and the injection of malicious nameserver metadata.

Overview. As shown in Figure 3, the attacker first sends ICMP Packet-Too-Big (PTB) messages to the authoritative nameserver (e.g., `ns1.vitm-ns.com`) of the victim domain (e.g., `vitm.com`) so that its responses to the resolver will fragment (①). Then, the attacker injects a forged second fragment, containing a fake IP address for the authoritative nameserver, into the target resolver (②). Next, the attacker triggers a lookup by causing the resolver to query an extremely long, non-existent subdomain under the nameserver domain (e.g., `[random].vitm-ns.com`) (③). Upon receiving a query, the recursive resolver performs an iterative query (④) and obtains a fragmented response from the nameserver (⑤). Since the attacker has pre-sent a forged second fragment into the buffer, once the legitimate first fragment arrives, the OS immediately re-assembles them. As a result, the legitimate first fragment and an attacker-injected second fragment are then reassembled by the resolver. Then, the attacker cannot forge a valid DNSSEC signature for the manipulated packet, and the resolver’s validation of the re-assembled response fails, but the record is saved in the BAD cache. Subsequently, when a client queries the resolver for `vitm.com`, the resolver directly uses the fake nameserver’s IP address stored in the BAD cache and sends the request to the attacker’s rogue nameserver (⑥). The rogue nameserver then returns a forged resource record for `vitm.com` (⑦). The rogue resource record is put into the positive cache and delivered to the client.

Fragmentation. The method follows existing fragment-based attacks [9, 25, 69]. The attacker first sends ICMP “Packet Too Big” messages—specifically IPv4 Type 3, Code 4 (Fragmentation Needed), with a reduced next-hop MTU toward the target authoritative server (e.g., `ns1.vitm-ns.com`), so that its oversized replies on the path to the resolver are fragmented according to that next-hop MTU. When the resolver receives a fragmented response from the authoritative nameserver, it reassembles the legitimate first fragment with a forged second fragment that was pre-injected by the attacker. Detailed fragmentation techniques are provided in the Appendix C. **Triggering large responses.** The packet sizes required for traditional fragmentation attacks [25] could be achieved with standard DNSSEC responses. However, as awareness of such attacks has

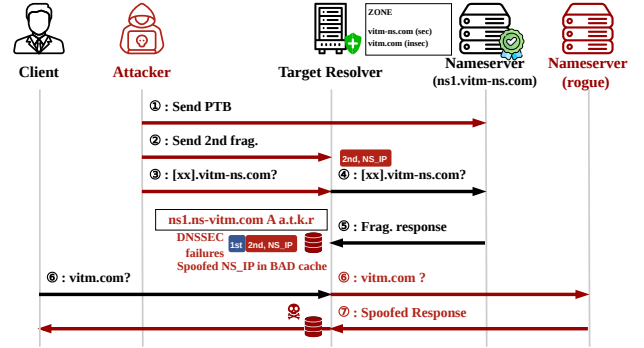
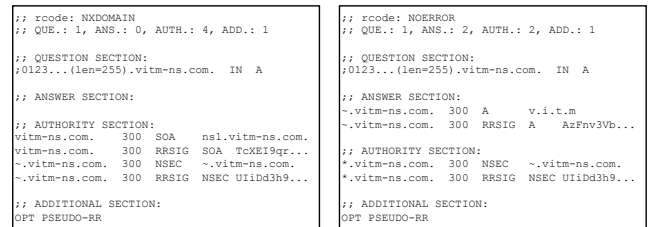


Figure 3: BADDNS attack workflow.



(a) Original DNS response: NXDOMAIN. (b) Original DNS response: NOERROR.

(c) Reassembled NXDOMAIN response: insert NS_IP. (d) Reassembled NXDOMAIN response: insert Glue.

Figure 4: Responses to queries for large non-existent names and response-crafting methods.

grown, the threshold for inducing fragmentation has been raised, and standard DNSSEC responses are often no longer large enough to meet this requirement. Therefore, we propose a novel technique that uses carefully crafted queries to elicit oversized responses from nameservers. Specifically, we trigger the resolver to ask for an *extremely long* domain name with EDNS and DNS Cookies—e.g., a chain of 63-character labels under `vitm-ns.com` that approaches the DNS limit of 255 octets per label. This design has three advantages: first, the QUESTION section is maximized by the near-limit QNAME; second, the signed domain name needs to produce a multi-RR reply (i.e., DNSSEC proof material); third, enabling EDNS with cookies deterministically yields exactly one OPT pseudo-resource record in the ADDITIONAL section, which gives one RR location for later modifications.

Our query may result in two possible responses: NOERROR response, if the domain configures the wildcard record, or NXDOMAIN

response (i.e., Name Error) if it does not. Figure 4 depicts the original, legitimate response and contrasts it with the response after being reassembled with our specially crafted second fragment. The NXDOMAIN response (Figure 4a) includes SOA/RRSIG and NSEC(NSEC3)/RRSIG in the AUTHORITY section, with an OPT RR in ADDITIONAL. The NOERROR response (Figure 4b) contains A+RRSIG in ANSWER and NSEC(NSEC3)+RRSIG in AUTHORITY, again with a single OPT in ADDITIONAL.

An excessively long query name is only one factor in amplifying the response size. Specific features of DNSSEC concerning non-existent domains and wildcard domains further contribute to inflating the response. For a non-existent domain (which results in NXDOMAIN), the nameserver is required to return at least four resource records, including two RRSIG records. Similarly, RFC 5155 [6] specifies that if a queried domain name matches a wildcard domain, NSEC/3 records must be returned. Consequently, for wildcard domains (which result in NOERROR), the authoritative server also returns at least four resource records, as illustrated in Figure 4b. **Injection of malicious nameserver metadata.** Querying our carefully crafted subdomains provides the ability to induce fragmentation, but it also prevents us from directly injecting A records for the nameserver into the ANSWER section. This is because, due to bailiwick restrictions, any resource record in the ANSWER section that does not match the queried domain name will be discarded. Therefore, we choose to modify the AUTHORITY and ADDITIONAL in the second fragment to inject fake resource records that resolvers later store in BAD cache, following the layouts in Figure 4. The lower half (panels (c)–(d)) illustrates, under the NXDOMAIN response, the two concrete modification methods we employ.

- **Insert NS_IP.** We replace the single OPT pseudo-RR in the ADDITIONAL section with an A RR for `ns1.vitm-ns.com` that points to an attacker-controlled IP (Figure 4c).
- **Insert Glue.** In addition to replacing OPT with an A RR for `ns1.vitm-ns.com`, we replace the last RRSIG in the AUTHORITY section with an NS RR, thereby injecting malicious resource records (Figure 4d).

Both modifications alter or remove the original RRSIG records and therefore cause DNSSEC validation to fail. However, resolvers still cache the resource records in AUTHORITY and ADDITIONAL sections into their BAD cache and then use them for future resolution, opening the door for practical exploitation scenarios.

4.3 Conditions of Practical Attacks

Driven by our threat model and the attack workflow, the target needs to satisfy the following conditions for BADDNS to succeed.

Resolvers with DNSSEC-enabled and BAD cache reuse. The target resolvers should correctly perform DNSSEC validation. When validation failures occur, the records should be admitted into a BAD cache and subsequently reused through the future resolving process, as described in Section 3.3. Since DNSSEC validation is already widely supported by mainstream resolver implementations and DNS service operators, the key is whether validation-failed records can be cached and later reused. To quantify this property in the wild and estimate the practical impact scope of BADDNS, we evaluated the real-world DNSSEC-validating public DNS providers and open resolvers in Section 6.1 and Section 6.2.

DNSSEC-signed authoritative responses that can be fragmented. The target name server (i.e., `ns1.vitm-ns.com`) should be signed with DNSSEC. Also, the DNS responses should be inducible to undergo IP fragmentation, enabling attackers to inject the BAD entry (i.e., `<ns1.vitm-ns.com A a.t.k.r>`) into the second fragment. In Section 6.3, we empirically assessed DNSSEC-signed authoritative servers in the wild by sending PTB messages to trigger fragmentation, then examined whether the observed second fragment admits performing *Insert NS_IP* or *Insert Glue*.

Unsigned domains hosted on the DNSSEC-signed authoritative nameserver. The victim domain hosted on the target authoritative nameserver should not be DNSSEC-enabled. Although the attack above already compromises the DNSSEC-enabled nameserver, its practical impact would be reduced to denial of service. Since the attacker cannot forge valid DNSSEC signatures, a validating resolver will reject the forged records. In contrast, for hosted domains that are not signed with DNSSEC, the attacker does not need to forge any signatures. The resolver can accept the forged resource records, admit them into the positive cache, and subsequently serve them to clients.

Prior works [12] studied the deployment of DNSSEC in the real world, and found a significant proportion of popular domains that do not deploy DNSSEC, while many DNS hosting services and TLD zones have signed DNSSEC. BADDNS benefits from today’s limited DNSSEC deployment, where hosting infrastructures are DNSSEC-signed while a large fraction of customer zones remain unsigned. In this setting, validating resolvers must still walk through the signed hosting nameservers to reach unsigned domains; if BAD-cache logic stores and later reuses delegation resource records that failed validation, unsigned dependents can be steered off-path without breaking any signatures. We did the real-world evaluation on this scenario in Section 6.3 and revealed that 35.87% of popular domains meet this condition. Because a single hosting NS typically serves many unsigned customer zones, one successful poisoning scales to dozens of domains behind it.

4.4 Comparison with Prior Attacks

Comparison with RUC attack. Zhang et al. [66] proposed a novel Denial-of-Service attack, utilizing how resolvers handle “troubleshooting” queries in a validation-disabled scenario. At a high level, both BADDNS and the RUC attack reveal a common design flaw: misuse of special-purpose caches in resolvers. However, the mechanisms and consequences diverge fundamentally. RUC abuses unvalidated troubleshooting data (introduced via CD=1) that lacks a clear specification and is therefore inconsistently handled across implementations, leading to persistent SERVFAIL DoS. In contrast, BADDNS exploits the BAD cache—an RFC-documented construct [54] for explicitly validating failed (bogus) data with prescribed usage restrictions. We show that mainstream resolvers reuse these *validation-failed* resources in subsequent resolution processes, which lets an off-path adversary steer lookups and hijack unsigned customer zones that delegate to DNSSEC-signed zones (Section 4.2). Thus, whereas RUC yields denial of service inside signed zones, BADDNS enables cache poisoning behind signed hosting at scale. **Comparison with prior cache poisoning attacks.** Classic cache-poisoning attacks against target resolvers [30, 36] lack sufficient

Table 2: End-to-end BADDNS attack results towards DNS software.

Resolver	Version	Name Error		Wildcard		TTL (s)	Restr.? [*]	Avg. Time (s)	Succ. Rate	Vuln.?
		Insert NS_IP	Insert Glue	Insert NS_IP	Insert Glue					
Knot Resolver [17]	6.0.14	✗	✓	✗	✗	86,400	✗	0.8	10/10	✓
PowerDNS Recursor [48]	5.2.2	✗	✗	✗	✓	86,400	✗	0.7	10/10	✓
Microsoft DNS [40]	2022	✓	✗	✓	✗	86,400	✗	0.9	10/10	✓

^{*} Whether the resolver enforces any usage restrictions on poisoned entries.

✓: Vulnerable. ✗: Not vulnerable. ✗: No.

entropy (TXID/port). Moreover, these poisoning attacks often presume domains that are *unsigned* or resolvers that *do not* validate DNSSEC. By contrast, BADDNS requires DNSSEC on the hosting nameserver and leverages the resolver’s handling of validation failures to poison unsigned delegations behind it. This “signed-hosting to unsigned-customer” condition is common in practice [11] and dramatically amplifies impact: one misused BAD-cache entry at the hosting layer can redirect many dependent, unsigned zones.

Fragmentation techniques. Our use of fragmentation is orthogonal to the above distinctions: we leave all randomized, entropy-bearing fields in the *first* fragment intact and modify only the *second* fragment (Authority/Additional). Unlike earlier work that assumed unrealistically tiny MTUs [9], our measurements in Section 6.3 show that over 98.99% of signed authorities fragment at practical path MTUs (528 bytes). Querying extremely long non-existent names reliably pushes OPT (and often the last RRSIG) into the second fragment, yielding a stable and feasible injection surface.

Summary. In contrast to prior works, BADDNS exposes a distinct, standards-adjacent weakness: validating resolvers that implement RFC-defined BAD cache may store and later reuse data that has explicitly failed DNSSEC validation, enabling off-path hijacking of unsigned customer zones that delegate to DNSSEC-signed hosting. Unlike the RUC attack, which abuses unspecified troubleshooting caches to cause persistent SERVFAIL, BADDNS achieves positive cache poisoning at scale under correct DNSSEC operation. Unlike classic forgery attacks, BADDNS does not guess entropy (TXID/port) and assumes the target resolvers can correctly do DNSSEC validation. Finally, unlike earlier fragmentation works, BADDNS assumes only practical MTUs, though adversaries still can modify the second fragment to inject the malicious records in DNSSEC-enabled contexts, making them once again feasible.

5 End-to-End Attack Feasibility

5.1 Experimental Design

We evaluated the effectiveness of BADDNS against three mainstream DNSSEC-validating resolvers that implement BAD cache: Microsoft DNS [40], PowerDNS Recursor [48], and Knot Resolver [17]. All targets were configured with a root trust anchor and enabled strict DNSSEC validation. To demonstrate real-world feasibility, we conducted attacks against domains whose authoritative servers were empirically confirmed to be vulnerable.

Environment setup. The tested authoritative servers were selected from the affected nameservers identified in Section 6.3. Consistent with the attack conditions in Section 4.3, these nameservers (e.g., *ns1.vitm-ns.com*) are properly configured with DNSSEC and

support fragmentation. At the same time, they host non-DNSSEC-signed domains (e.g., *vitm.com*). We validate the effectiveness of BADDNS by attacking the resolvers controlled by us and checking whether the cache entry of *ns1.vitm-ns.com* is successfully poisoned.

We exercised two hosting configurations in turn. In the *Name Error* configuration, only *ns1.vitm-ns.com* existed, so that other subdomains under *vitm-ns.com* returned NXDOMAIN with DNSSEC proofs. In the *Wildcard* configuration, a wildcard RRset was present so that queries under *vitm-ns.com* yielded A+RRSIG in the ANSWER section and non-existence proofs in AUTHORITY.

For each configuration, we applied the two injection methods introduced in 4.2. In *Insert NS_IP*, the OPT in ADDITIONAL was replaced with an A RR for *ns1.vitm-ns.com* that pointed to the attacker-controlled nameserver. In *Insert Glue*, we replaced OPT with that A RR and additionally replaced the last RRSIG in AUTHORITY with an NS RR, as shown in Figure 4d, creating a forged delegation pair.

Experimental step. We first validated whether the resource records from our injected responses, which are designed to fail DNSSEC validation, were cached and subsequently used in future resolution processes. Specifically, once the record injection was complete—that is, after the recursive resolver had received the validation-failing response—we initiated a query for the domain *www.vitm.com*. We then observed whether the resolver was injected with the forged resource record, which is confirmed if it sends a subsequent query to the attacker-controlled rogue nameserver. Concurrently, we manually inspected the resolver’s cache to confirm whether the forged record was stored and to determine its assigned TTL value.

Given the specialized nature of the BAD cache, we also need to verify if any additional restrictions are imposed when its data is used in other processes, as this could impact the efficacy of our attack. According to the protocol design, the BAD cache is intended for use only in DNSSEC validation scenarios, and its TTL is meant to be strictly limited. Our testing, therefore, focuses on two aspects: first, whether a subsequent legitimate-looking response from our rogue server is stored in the standard positive cache; and second, if so, whether the resource record’s TTL is modified. To do this, we configure our rogue nameserver to return a standard resource record for *www.vitm.com* and manually inspect the resolver’s cache.

5.2 Results

As summarized in Table 2, all three of the analyzed resolver implementations that utilize the BAD cache were found to be vulnerable to the BADDNS attack. They were each susceptible to at least one attack payload and, critically, would completely trust the injected forged resource records, reusing them in subsequent resolutions

without imposing any additional restrictions. The average time cost of BADDNS is lower than 1 second, and all the test cases are successful.

Across configurations, we consistently observe that each resolver accepts at least one of the two modifications into the BAD cache and subsequently consults that poisoned state during delegation. Concretely, the *Insert Glue* modification is effective when combined with *Name Error* on Knot Resolver, and with *Wildcard* on PowerDNS Recursor; by contrast, *Insert NS_IP* is the variant that succeeds on Microsoft DNS, and it does so under both *Name Error* and *Wildcard*. These outcomes indicate that different implementations expose distinct acceptance surfaces for validation-failed authority/additional metadata, but in every case, there exists at least one practical path to seed the BAD cache and trigger malicious redirection.

The resolver ignores the distinctions between its different cache types, to the extent that it will completely trust data for a name-server sourced from the BAD cache. In all successful cases, the resolver stores the attacker's A response for `www.vitm.com` in the *positive* cache, which can be directly retrieved by arbitrary clients with NOERROR. We also observed that no enforcement ties the lifetime of this positive entry to the limited TTL used for BAD cache. Instead, the effective TTL tracks the attacker-advertised value, on the order of 86,400 seconds in our experiments, demonstrating that BAD cache serves as the trigger but does not constrain the persistence of the poisoned positive cache entry.

6 Real-world Evaluation

6.1 Public DNS Services

Dataset. Guided by APNIC statistics [5], we first compiled a list of widely used public DNS services. We then tested their support for DNSSEC and ultimately selected 14 of these providers for our evaluation, as listed in Table 3.

Methodology. We followed the same experimental setup that was detailed in Section 5.1, a controlled, DNSSEC-signed nameserver (`ns1.vitm-ns.com`) serving an unsigned customer domain (`vitm.com`). For the victim nameserver, we tested two authoritative configurations with two modification variants. Moreover, we configured our rogue authoritative nameserver to serve a resource record for the victim domain (i.e., `www.vitm.com`) that was distinct from the one on the legitimate authoritative server. This setup allows us to directly infer the resolver's behavior by comparing the resource record received by the client. To cope with anycast and back-end heterogeneity [64], we ran multiple rounds per IP and confirmed the outcome in repeated trials.

Results. As summarized in Table 3, 10/14 providers are vulnerable in at least one attack scenario. In the *Wildcard* scenario, the *Insert Glue* attack vector is the most effective, impacting nine service providers, including Oracle Dyn, Quad9, and UltraDNS. In contrast, under the *Name Error* (i.e., NXDOMAIN) scenario, only two service providers are affected. CZ.NIC DNS is only vulnerable to the *Insert Glue* attack within the *Name Error* scenario, whereas OpenDNS is found to be susceptible to all tested attack payloads. After poisoning succeeds, the forged record is consistently placed into positive cache with long persistence—TTL values cluster around 86,400s, with provider-specific clamps (e.g., 3,600s for Dyn; 43,200s for Neustar,

Quad9, and UltraDNS)—indicating that BAD cache is used only as a trigger and does not constrain the lifetime of the poisoned entry.

6.2 Open DNS Resolvers

Dataset. To perform a large-scale impact assessment on in-the-wild open resolvers, we leveraged ZMap [22] to scan the routable IPv4 space for hosts listening on UDP port 53, yielding 774,099 open resolvers. Next, we tested their support for DNSSEC. Specifically, we configured two subdomains under a domain we control: one with a valid DNSSEC signature and another with an invalid signature. We then sent queries for both subdomains to each open resolver and determined its DNSSEC support status by analyzing the responses. A resolver was classified as having DNSSEC validation enabled if it returned a valid response for the correctly signed subdomain but an error (e.g., SERVFAIL) for the incorrectly signed one; otherwise, it was classified as not having validation enabled. Furthermore, to mitigate the impact of transient network issues such as latency, we repeated this test three times for each resolver. A resolver was marked as DNSSEC-validating if it passed the validation check in at least one of the three attempts. Finally, we found 300,792 open resolvers that enable DNSSEC validation.

We then evaluated each open resolver using the same configuration and procedure described in Section 6.1.

Results. Table 4 summarizes our test results. Among the 300,792 DNSSEC-validating open resolvers we measured, 50,712 (16.86%) were successfully steered to the attacker's nameserver and accepted the attacker's A answer for `www.vitm.com` into their positive cache. The most effective attack setting was the *Wildcard* + *Insert Glue* variant (40,923, 13.60%). Our experimental results confirm that a large number of resolvers conflate the usage boundaries of different cache types. They improperly trust and utilize data from the BAD cache—containing records from failed DNSSEC validations—in future resolution processes, which demonstrates the widespread impact of the BADDNS attack.

6.3 Vulnerable Nameservers and Domains

Dataset. To evaluate the real-world impact of BADDNS, we selected the Tranco [46] Top 1 Million domains for our analysis. We first used ZDNS [29] to collect the nameservers for these domains, resulting in a set of 266,680 unique servers. Subsequently, we filtered this set to retain only those nameservers that are DNSSEC-enabled. For each nameserver, we issued A queries with DNSSEC and checked if the response carries RRSIG as a signed zone. This filtering produced 26,930 DNSSEC-signed nameservers that host a total of 358,790 customer domains.

Methodology. Our goal is as follows. For each signed nameserver, can its DNS responses be induced into fragments? And, if so, where does the split point land relative to DNS sections that matter to BAD data injection in BADDNS?

For each nameserver, we first sent queries that are crafted to elicit the largest legitimate response without triggering fragmentation: a QNAME that approaches the 255-octet DNS limit (as introduced in Section 4) under one of the zones hosted by the nameserver, with DO bit, EDNS, and DNS Cookies enabled. We analyzed the size of the whole DNS response without fragmentation, the start offset of the OPT pseudo-RR in ADDITIONAL, and the start offset

Table 3: BADDNS attacks towards public DNS services.

Resolver	IP	Name Error		Wildcard		TTL (s)	Vuln.?
		Insert NS_IP	Insert Glue	Insert NS_IP	Insert Glue		
Google [20]	8.8.8.8	✗	✗	✗	✗	–	✗
	8.8.4.4	✗	✗	✗	✗	–	✗
Cloudflare [15]	1.1.1.1	✗	✗	✗	✗	–	✗
	1.0.0.1	✗	✗	✗	✗	–	✗
DNS.WATCH [21]	84.200.69.80	✗	✗	✗	✓	86,400	✓
	84.200.70.40	✗	✗	✗	✓	86,400	✓
Dyn [23]	216.146.35.35	✗	✗	✗	✓	3,600	✓
	216.146.36.36	✗	✗	✗	✓	3,600	✓
OpenDNS [44]	208.67.222.222	✓	✓	✓	✓	86,400	✓
	208.67.220.220	✓	✓	✓	✓	86,400	✓
Neustar [42]	156.154.70.1	✗	✗	✗	✓	43,200	✓
	156.154.71.1	✗	✗	✗	✓	43,200	✓
Quad9 [52]	9.9.9.9	✗	✗	✗	✓	43,200	✓
	149.112.112.112	✗	✗	✗	✓	43,200	✓
UltraDNS [61]	64.6.64.6	✗	✗	✗	✓	43,200	✓
	64.6.65.6	✗	✗	✗	✓	43,200	✓
Comodo Secure DNS [19]	8.26.56.26	✗	✗	✗	✓	86,400	✓
	8.20.247.20	✗	✗	✗	✓	86,400	✓
CleanBrowsing [14]	185.228.168.9	✗	✗	✗	✓	86,400	✓
Quad 101 [60]	101.101.101.101	✗	✗	✗	✗	–	✗
	101.102.103.104	✗	✗	✗	✗	–	✗
AdGuard DNS [1]	94.140.14.14	✗	✗	✗	✓	86,400	✓
	94.140.15.15	✗	✗	✗	✓	86,400	✓
Canadian Shield [13]	149.112.121.10	✗	✗	✗	✗	–	✗
	149.112.122.10	✗	✗	✗	✗	–	✗
CZ.NIC DNS [16]	193.17.47.1	✗	✓	✗	✗	86,400	✓
	185.43.135.1	✗	✓	✗	✗	86,400	✓

✓: Vulnerable. ✗: Not vulnerable.

Table 4: BADDNS attacks towards DNSSEC-validating open resolvers.

Category	Count	Prop.
DNSSEC-validating open resolvers	300,792	100.00%
<i>Name Error</i>		
Insert Glue	19,467	6.47%
Insert NS_IP	9,939	3.30%
<i>Wildcard</i>		
Insert Glue	40,923	13.60%
Insert NS_IP	8,752	2.91%
Total poisoned	50,712	16.86%

of the last RRSIG in AUTHORITY. We also classified the behavior of the nameserver as Name Error (responding with NXDOMAIN) or Wildcard (responding with NOERROR).

Next, we attempted to induce fragmentation in the responses from the authoritative nameservers. For each target nameserver, we sent an IPv4 ICMP PTB message with a 68 next-hop MTU, then immediately queried the nameserver with the same method as the former step. We captured the first UDP fragment (MF=1, offset=0) arriving from the tested nameserver and recorded the size of its IP

payload. The nameserver that does not support fragmentation was considered not affected.

Furthermore, we analyzed where the IP-payload split point lands within the oversized DNS response to identify the exploitable cases. For responses that did fragment, we observed three regimes: (i) the split occurs *before* the start offset of the last RRSIG in the AUTHORITY section; (ii) the split falls *after* that last RRSIG but *before* the start offset of the OPT pseudo-RR in ADDITIONAL; and (iii) the split is *after* the start of OPT. Case (1) leaves the entire RRSIG of AUTHORITY and all of ADDITIONAL in the second fragment, enabling *both* modification strategies—either *Insert Glue* (replace the trailing RRSIG with an NS RR and pair it with an address in ADDITIONAL) or *Insert NS_IP* (overwrite OPT with an A RR for ns1.vi tm-ns.com). Case (2) guarantees that OPT resides in the second fragment but not the final RRSIG, so only *Insert NS_IP* is feasible. Case (3) places the cut in the middle of OPT, leaving no modifiable authority/additional RRs in the second fragment, and thus offers no practical injection surface.

To ensure that no middleboxes interfered with our measurements, we conducted the above measurement from multiple, geographically distributed vantage points and repeated each test multiple times. Specifically, we established five vantage points, including

Table 5: Results of DNSSEC-signed Nameserver Tests.

	Wildcard	Name Error	Total
Dataset			
Signed NS	4,013	22,917	26,930
Signed Frag. NS ¹	1,974	7,081	9,055
Split Position			
(i) Before RRSIG	1,959	6,077	8,036
(ii) Between RRSIG & OPT	12	615	627
(iii) After OPT	3	389	392
Results			
Affected NS ²	1,971	6,692	8,663
Affected Domains ³	13,815	6,656	20,392

¹ The count of DNSSEC-signed nameservers that can fragment under PTB.

² The count of nameservers whose split position is either (i) *before* the last RRSIG in AUTHORITY or (ii) *between* the last RRSIG and the OPT record.

³ The count of unsigned domains delegated to the affected nameservers.

locations in the United States, Hong Kong, and Singapore. Then, we ran three trials from each vantage point and conservatively classified a nameserver as supporting fragmentation only if fragmented responses were consistently observed across all trials.

For each vulnerable nameserver, we then evaluated the scope of its impact. We sent queries to the domains served by these nameservers and determined each domain’s DNSSEC deployment status by checking for the presence of an RRSIG resource record in the response. Domains without such an RRSIG were classified as unsigned and thus affected by BADDNS.

Fragmentation MTU in the wild. Using ICMP PTB probing, we measured the first-fragment IP-payload size for 26,930 signed nameservers, as shown in Table 5. Most nameservers did not fragment under our PTB trigger (17,875, 66.38%). Among the 9,055 that did fragment, the distribution is sharply concentrated: 8,050 (88.90% of fragmenters; 29.89% overall) cut at 528 bytes of IP payload for the first fragment; 914 (10.09% of fragmenters; 3.39% overall) cut at 1,480 bytes; all other cut sizes together account for the remaining 1.01% of fragmenters.

Affected nameservers and domains. Table 5 summarizes where the IP-payload split point lands within the DNS message. Out of 26,930 signed nameservers, 9,055 (33.62%) can be induced to generate fragmented UDP responses. Of these, 8,036 (88.75%) split *before* the last RRSIG in AUTHORITY, and 627 (6.92%) split *between* that RRSIG and the OPT record. Because the two regimes admit second-fragment modification, we count 8,663 nameservers as practically exploitable, including Cloudflare. Broken down by configuration, the Wildcard contributes 1,971 affected NS, and the Name Error contributes 6,692. In addition, our results show that 20,392 (5.68%) domains are vulnerable to BADDNS. Among these, 1,320 domains are ranked in the top 100k, including prominent examples such as the leading global e-commerce platform Shopify [56] and popular marketplace for free Android applications APKPure [3].

7 Discussion

7.1 Mitigation

Strict semantics for BAD cache. The BAD cache was added as an optional optimization, which leaves room for under-specification

and divergent vendor interpretations. Based on our analysis of mainstream implementations, we recommend the following concrete design rules.

- **Entry content.** BADDNS is enabled by reusing data from failed validations during later non-DNSSEC resolution steps. To eliminate this channel, resolvers should not cache full RRsets from failed validations. Instead, cache only a tuple such as $\langle \text{qname}, \text{qclass}, \text{qtype} \rangle$ annotated as *BAD* and use it solely to throttle revalidation traffic. Functionally, this turns BAD cache into a strictly rate-limiting mechanism akin to negative caching, never a source of resolvable data.

- **TTL discipline.** Even though our attack needs only a small injection window, we observed that several resolvers reuse the original (untrustworthy) TTL or impose very loose limits. Upon detecting a DNSSEC validation failure, implementations should overwrite the TTL with a short, hard cap (e.g., 60s). This upper bound must not be extendable by any record when validation fails.

- **Cache partitioning and policy.** Most implementations we studied do not clearly segregate material derived from failed validations. Validation-failed entries flow into the same store as positive data. Implementations should explicitly partition caches and tag provenance. Entries marked *BAD* must (i) never be promoted to, or coalesced with, the positive cache, (ii) never influence delegation choices (e.g., glue/NS address selection), and (iii) never be served to clients. The only permitted action is to suppress retries until the short TTL expires or a trust context changes.

DNSSEC usage policy for hosting. BADDNS succeeds when an unsigned customer zone delegates to a DNSSEC-signed hosting infrastructure. Hosting setups where a DNSSEC-signed infrastructure serves many unsigned customer zones are inherently risky; providers should first drive toward consistent signing (turn-key zone signing and automated DS publication for customers). In parallel, providers can shrink the attack surface by controlling response size: (i) prefer compact algorithms such as ECDSA-P256 or Ed25519 over RSA to minimize RRSIG [27]; and (ii) when a response is predicted to exceed a safe threshold (e.g., 528 bytes), return a minimal UDP reply without signatures and set the TC bit to force TCP fallback. This combination reduces fragmentation opportunities while preserving DNSSEC correctness over TCP, where large proof material can be transferred reliably.

7.2 Responsible Disclosure

With the established ethical guidelines for vulnerability research, we conducted responsible disclosure of BADDNS in July 2025 to all affected vendors. We’ve built the PoC introduced in Appendix B, which has already been used by multiple vendors for reproduction.

Our disclosure timeline and responses are summarized below:

- **PowerDNS Recursor.** We provided a detailed report of the vulnerability and proposed mitigations. PowerDNS has acknowledged our report and the impact of BADDNS. Because changing the BAD cache structure would require substantial refactoring, PowerDNS chose a lighter mitigation by strengthening delegation validation, preventing attackers from injecting BAD cache via *Insert Glue*. So far, PowerDNS has released the patch [47] and assigned a CVE.

- **Microsoft DNS.** We submitted a full report together with a step-by-step reproduction video. Microsoft confirmed BAD cache

exploitation and is currently coordinating with PowerDNS on mitigation and disclosure.

• **Knot Resolver.** We contacted the Knot team with our findings. They have reviewed the report and begun their internal assessment.

• **AdGuard, OpenDNS, and UltraDNS.** For the public DNS services we found to be affected, we disclosed our findings and supplied proof-of-concept (PoC) artifacts to aid reproduction. Among them, AdGuard is reproducing the issue, OpenDNS is reviewing the report, and UltraDNS has acknowledged the issue.

• **Cloudflare.** For the DNS hosting provider we identified as affected, we disclosed our findings and provided PoC materials and the vulnerable nameserver list to facilitate verification. Among them, Cloudflare has reviewed the report, reproduced the attack, and is now discussing mitigation strategies.

8 Conclusion

This paper reveals the impact of DNSSEC on unsigned domains and discloses a novel cache poisoning attack BADDNS, stemming from the improper use of DNSSEC validation-failure resource records. We demonstrated that 3 major resolver implementations are vulnerable, including Microsoft DNS, PowerDNS, and Knot, as they incorrectly apply the BAD cache to future resolution processes. Large-scale evaluations show that 16.86% of DNSSEC-validating open resolvers and 10 public DNS providers, including OpenDNS and Quad9, are vulnerable to BADDNS. Furthermore, our assessment of the Tranco Top 1M domains revealed that one-third of the properly configured, DNSSEC-enabled nameservers are vulnerable, affecting 5.68% of the domains they serve (e.g., Shopify and AP-KPure). We have responsibly disclosed our findings to all affected providers. To date, 7 of these vendors have responded positively, including PowerDNS, and some are in the process of implementing patches.

Acknowledgments

We thank all anonymous reviewers for their valuable and constructive feedback. This work is supported in part by the National Key Research and Development Program of China (No. 2023YFB3105600), Zhongguancun Laboratory and the National Natural Science Foundation of China (62102218). Yunyi Zhang is partially supported by the Shuimu Tsinghua Scholar Program.

References

- [1] AdGuard. 2025. Connect to public AdGuard DNS server. <https://adguard-dns.io/en/public-dns.html>
- [2] Mark P. Andrews. 1998. Negative Caching of DNS Queries (DNS NCACHE). RFC 2308. doi:10.17487/RFC2308
- [3] APKPure. 2025. Free APK Download on Android - Download Apps with Free Online APK Downloader. <https://apkpure.net/>
- [4] APNIC. 2025. DNSSEC Validation Rate by country. <https://stats.labs.apnic.net/dnssec>
- [5] APNIC. 2025. Use of DNS Resolvers for World. <https://stats.labs.apnic.net/rvrs>
- [6] Roy Arends, Geoffrey Sisson, David Blacka, and Ben Laurie. 2008. DNS Security (DNSSEC) Hashed Authenticated Denial of Existence. RFC 5155. doi:10.17487/RFC5155
- [7] Michael D. Bailey, David Dittrich, Erin Kenneally, and Douglas Maughan. 2012. The Menlo Report. *IEEE Secur. Priv.* 10, 2 (2012), 71–75. doi:10.1109/MSP.2012.52
- [8] BIND9. 2025. `dns_badcache_add()`. <https://gitlab.isc.org/isc-projects/bind9/-/blob/v9.21.9/lib/dns/badcache.c#L251>
- [9] Markus Brandt, Tianxiang Dai, Amit Klein, Haya Shulman, and Michael Waidner. 2018. Domain Validation++ For MitM-Resilient PKI. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) (CCS '18). Association for Computing Machinery, New York, NY, USA, 2060–2076. doi:10.1145/3243734.3243790
- [10] CAIDA. 2025. State of IP Spoofing. <https://spoofer.caida.org/summary.php>
- [11] Taejoong Chung, Roland van Rijswijk-Deij, Balakrishnan Chandrasekaran, David Choffnes, Dave Levin, Bruce M. Maggs, Alan Mislove, and Christo Wilson. 2017. A longitudinal, end-to-end view of the DNSSEC ecosystem. In *Proceedings of the 26th USENIX Conference on Security Symposium* (Vancouver, BC, Canada) (SEC'17). USENIX Association, USA, 1307–1322.
- [12] Taejoong Chung, Roland van Rijswijk-Deij, David Choffnes, Dave Levin, Bruce M. Maggs, Alan Mislove, and Christo Wilson. 2017. Understanding the role of registrars in DNSSEC deployment. In *Proceedings of the 2017 Internet Measurement Conference* (London, United Kingdom) (IMC '17). Association for Computing Machinery, New York, NY, USA, 369–383. doi:10.1145/3131365.3131373
- [13] CIRA. 2020. How CIRA Canadian Shield works – CIRA. <https://www.cira.ca/en/how-canadian-shield-works/>
- [14] CleanBrowsing. 2025. CleanBrowsing | DNS Filtering for Schools, Families, and Businesses. <https://cleanbrowsing.org/>
- [15] Cloudflare. 2025. 1.1.1.1 — One of the Internet's Fastest, Privacy-First DNS Resolver. <https://one.one.one.one/dns/>
- [16] CZ.NIC. 2025. CZ.NIC - Open DNSSEC Validating Resolvers. <https://www.nic.cz/odvr/>
- [17] CZ.NIC. 2025. Knot Resolver. <https://www.knot-resolver.cz/>
- [18] Tianxiang Dai, Haya Shulman, and Michael Waidner. 2016. DNSSEC Misconfigurations in Popular Domains. In *Cryptography and Network Security: 15th International Conference, CANS 2016, Milan, Italy, November 14-16, 2016, Proceedings* (Milan, Italy). Springer-Verlag, Berlin, Heidelberg, 651–660. doi:10.1007/978-3-319-48965-0_43
- [19] Comodo Secure DNS. 2025. Free Comodo Secure DNS | Best DNS Security Solution. <https://www.comodo.com/secure-dns/>
- [20] Google DNS. 2025. Public DNS | Google for Developers. <https://developers.google.com/speed/public-dns>
- [21] DNS.WATCH. 2025. Fast, free and uncensored. - DNS.WATCH. <https://dns.watch/>
- [22] Zakir Durumeric, Eric Wustrow, and J. Alex Halderman. 2013. ZMap: Fast Internet-wide Scanning and Its Security Applications. In *Proceedings of the 22th USENIX Security Symposium, Washington, DC, USA, August 14-16, 2013*, Samuel T. King (Ed.). USENIX Association, 605–620. <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/paper/durumeric>
- [23] Dyn. 2025. Internet Guide Setup Dyn Help Center. <https://help.dyn.com/internet-guide-setup.html>
- [24] Elias Heftrig, Haya Shulman, and Michael Waidner. 2023. Downgrading DNSSEC: how to exploit crypto agility for hijacking signed zones. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) (SEC '23). USENIX Association, USA, Article 416, 16 pages.
- [25] Amir Herzberg and Haya Schulmann. 2013. Fragmentation Considered Poisonous, or: One-domain-to-rule-them-all.org. In *IEEE Conference on Communications and Network Security, CNS 2013, National Harbor, MD, USA, October 14-16, 2013*. IEEE, 224–232. doi:10.1109/CNS.2013.6682711
- [26] Bert Hubert and Remco Mook. 2009. Measures for Making DNS More Resilient against Forged Answers. RFC 5452. doi:10.17487/RFC5452
- [27] Geoff Huston. 2021. DNSSEC with RSA-4096 keys | APNIC Blog. <https://blog.apnic.net/2021/10/18/dnssec-with-rsa-4096-keys/>
- [28] ISC. 2025. BIND 9. <https://www.isc.org/bind/>
- [29] Liz Izhikevich, Gautam Akiwate, Briana Berger, Spencer Drakontaidis, Anna Ascherman, Paul Pearce, David Adrian, and Zakir Durumeric. 2022. ZDNS: a fast DNS toolkit for internet measurement. In *Proceedings of the 22nd ACM Internet Measurement Conference, IMC 2022, Nice, France, October 25-27, 2022*, Chadi Barakat, Cristel Pelsser, Theophilus A. Benson, and David R. Choffnes (Eds.). ACM, 33–43. doi:10.1145/3517745.3561434
- [30] Dan Kaminsky. 2008. Black ops 2008: It's the end of the cache as we know it. *Black Hat USA 2* (2008).
- [31] Knot. 2025. `answer_from_pkt()`. https://gitlab.nic.cz/knot/knot-resolver/-/blob/v6.0.14/lib/cache/entry_pkt.c#L196
- [32] Knot. 2025. `stash_pkt()`. https://gitlab.nic.cz/knot/knot-resolver/-/blob/v6.0.14/lib/cache/entry_pkt.c?ref_type=tags#L42
- [33] Knot. 2025. `validate_section()`. https://gitlab.nic.cz/knot/knot-resolver/-/blob/v6.0.14/lib/layer/validate.c?ref_type=tags#L243
- [34] Tadayoshi Kohno, Yasemin Acar, and Wulf Loh. 2023. Ethical frameworks and computer security trolley problems: foundations for conversations. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) (SEC '23). USENIX Association, USA, Article 288, 18 pages.
- [35] Xiang Li, Chaoyi Lu, Baojun Liu, Qifan Zhang, Zhou Li, Haixin Duan, and Qi Li. 2023. The maginot line: attacking the boundary of DNS caching protection. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) (SEC '23). USENIX Association, USA, Article 177, 18 pages.
- [36] Xiang Li, Wei Xu, Baojun Liu, Mingming Zhang, Zhou Li, Jia Zhang, Deliang Chang, Xiaofeng Zheng, Chuhan Wang, Jianjun Chen, Haixin Duan, and Qi Li. 2024. TuDoor Attack: Systematically Exploring and Exploiting Logic Vulnerabilities in DNS Response Pre-processing with Malformed Packets. In *IEEE*

- Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 4459–4477. doi:10.1109/SP54263.2024.00172
- [37] Xiang Li, Mingming Zhang, Zuyao Xu, Fasheng Miao, Yuqi Qiu, Baojun Liu, Jia Zhang, Xiaofeng Zheng, Haixin Duan, Zheli Liu, Yunhai Zhang, and Dunqiu Fan. 2025. RebirthDay Attack: Reviving DNS Cache Poisoning with the Birthday Paradox. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*, Chun-Ying Huang, Jyh-Cheng Chen, Shiuh-Pyng Shieh, David Lie, and Véronique Cortier (Eds.). ACM, 1619–1633. doi:10.1145/3719027.3744832
- [38] Keyu Man, Zhiyun Qian, Zhongjie Wang, Xiaofeng Zheng, Youjun Huang, and Haixin Duan. 2020. DNS Cache Poisoning Attack Reloaded: Revolutions with Side Channels. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (Virtual Event, USA) (CCS '20)*. Association for Computing Machinery, New York, NY, USA, 1337–1350. doi:10.1145/3372297.3417280
- [39] Keyu Man, Xin'an Zhou, and Zhiyun Qian. 2021. DNS Cache Poisoning Attack: Resurrections with Side Channels. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (Virtual Event, Republic of Korea) (CCS '21)*. Association for Computing Machinery, New York, NY, USA, 3400–3414. doi:10.1145/3460120.3486219
- [40] Microsoft. 2025. Domain Name System (DNS) in Windows and Windows Server. <https://learn.microsoft.com/en-us/windows-server/networking/dns/dns-overview>
- [41] Md National Commission for the Protection of Human Subjects of Biomedical and Behavioral Research, Bethesda. 1978. *The Belmont report: Ethical principles and guidelines for the protection of human subjects of research*. Superintendent of Documents.
- [42] Neustar. 2021. DNS Hosting Services for Business | Neustar. <https://www.home.neustar/dns-services>
- [43] NLnetLabs. 2025. Unbound. <https://nlnetlabs.nl/projects/unbound/about/>
- [44] OpenDNS. 2025. Cloud Delivered Enterprise Security by OpenDNS. <https://www.opendns.com/>
- [45] Craig Partridge and Mark Allman. 2016. Ethical considerations in network measurement papers. *Commun. ACM* 59, 10 (2016), 58–64.
- [46] Victor Le Pochat, Tom van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczynski, and Wouter Joosen. 2019. Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24-27, 2019*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/tranco-a-research-oriented-top-sites-ranking-hardened-against-manipulation/>
- [47] PowerDNS. 2025. Changelogs for 5.3.X. <https://github.com/PowerDNS/pdns/blob/master/pdns/recursor/docs/changelog/5.3.rst#id1>
- [48] PowerDNS. 2025. PowerDNS Recursor. <https://www.powerdns.com/powerdns-recursor>
- [49] PowerDNS. 2025. SyncRes::doCacheCheck(). https://gitlab.nic.cz/knot/knot-resolver/-/blob/v6.0.14/lib/cache/entry_pkt.c#L196
- [50] PowerDNS. 2025. SyncRes::updateValidationStatusInCache(). https://gitlab.nic.cz/knot/knot-resolver/-/blob/v6.0.14/lib/cache/entry_pkt.c?ref_type=tags#L42
- [51] PowerDNS. 2025. SyncRes::validateDNSKeys(). https://gitlab.nic.cz/knot/knot-resolver/-/blob/v6.0.14/lib/layer/validate.c?ref_type=tags#L243
- [52] Quad9. 2025. Quad9 | A public and free DNS service for a better security and privacy. <https://quad9.net/>
- [53] Scott Rose, Matt Larson, Dan Massey, Rob Austein, and Roy Arends. 2005. DNS Security Introduction and Requirements. RFC 4033. doi:10.17487/RFC4033
- [54] Scott Rose, Matt Larson, Dan Massey, Rob Austein, and Roy Arends. 2005. Protocol Modifications for the DNS Security Extensions. RFC 4035. doi:10.17487/RFC4035
- [55] Scott Rose, Matt Larson, Dan Massey, Rob Austein, and Roy Arends. 2005. Resource Records for the DNS Security Extensions. RFC 4034. doi:10.17487/RFC4034
- [56] Shopify. 2025. Shopify: The All-in-One Commerce Platform for Businesses. <https://www.shopify.com/?country=us&lang=en>
- [57] Soel Son and Vitaly Shmatikov. 2010. The Hitchhiker's Guide to DNS Cache Poisoning. In *Security and Privacy in Communication Networks - 6th International ICST Conference, SecureComm 2010, Singapore, September 7-9, 2010. Proceedings (Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, Vol. 50)*, Sushil Jajodia and Jianying Zhou (Eds.). Springer, 466–483.
- [58] Technitium. 2025. RecursiveResolverBackgroundTaskAsync(). <https://github.com/TechnitiumSoftware/DnsServer/blob/767f764f088d9ea7df065277aa0a2a2c8352f3/DnsServerCore/Dns/DnsServer.cs#L3372>
- [59] Technitium. 2025. Technitium DNS Server. <https://technitium.com/dns/>
- [60] TWINC. 2025. Quad 101. https://101.101.101.101/index_en.html
- [61] UltraDNS. 2025. UltraDNS Public | Vercara. <https://vercara.digicert.com/ultra-dns-public>
- [62] Unbound. 2025. already_validated(). <https://github.com/NLnetLabs/unbound/blob/30c13d0351abd2edc3d6dc76365f576c87b9736e/validator/validator.c#L446>
- [63] Sam Weiler and David Blacka. 2013. Clarifications and Implementation Notes for DNS Security (DNSSEC). RFC 6840. doi:10.17487/RFC6840
- [64] Wei Xu, Xiang Li, Chaoyi Lu, Baojun Liu, Haixin Duan, Jia Zhang, Jianjun Chen, and Tao Wan. 2023. TsuKing: Coordinating DNS Resolvers and Queries into Potent DoS Amplifiers. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (Copenhagen, Denmark) (CCS '23)*. Association for Computing Machinery, New York, NY, USA, 311–325. doi:10.1145/3576915.3616668
- [65] Yuxiao Wu, Yunyi Zhang, Chaoyi Lu and Baojun Liu. 2026. Should I Trust You? Rethinking the Principle of Zone-Based Isolation DNS Bailiwick Checking. In *Proceedings of the 33th Annual Network and Distributed System Security Symposium (NDSS 2026)*.
- [66] Shuhan Zhang, Shuai Wang, Li Chen, Dan Li, and Baojun Liu. 2025. Your Shield is My Sword: A Persistent Denial-of-Service Attack via the Reuse of Unvalidated Caches in DNSSEC Validation. In *Proceedings of the 34th USENIX Conference on Security Symposium (Seattle, WA, USA) (SEC '25)*. USENIX Association, USA, Article 416, 16 pages.
- [67] Yunyi Zhang, Baojun Liu, Haixin Duan, Min Zhang, Xiang Li, Fan Shi, Chengxi Xu, and Eihai Alowaischq. 2024. Rethinking the security threats of stale DNS glue records. In *Proceedings of the 33rd USENIX Conference on Security Symposium (Philadelphia, PA, USA) (SEC '24)*. USENIX Association, USA, Article 71, 17 pages.
- [68] Yunyi Zhang, Xikai Xiong, Baojun Liu, and Haixin Duan. 2026. Dart: An Automated and Reproducible Environment Toolkit for DNS Protocol Analysis. arXiv:2608.04498 [cs.NI] <https://arxiv.org/abs/2608.04498>
- [69] Xiaofeng Zheng, Chaoyi Lu, Jian Peng, Qiushi Yang, Dongjie Zhou, Baojun Liu, Keyu Man, Shuang Hao, Haixin Duan, and Zhiyun Qian. 2020. Poison over troubled forwards: a cache poisoning attack targeting DNS forwarding device. In *Proceedings of the 29th USENIX Conference on Security Symposium (SEC'20)*. USENIX Association, USA, Article 33, 17 pages.

A Ethical Considerations

Our study investigates how BAD cache is implemented and (mis)used in DNSSEC-validating resolvers and evaluates the real-world exposure of a resulting cache-poisoning threat. We followed ethical frameworks in computer security research [34], ethical principles [7, 41], and community best practices for network measurement and vulnerability research [45]. Below, we explain potential risks, our harm-minimization measures, and disclosure practices.

Closed-network experiments. DNS implementation studies on BAD cache in Section 3.1 were performed in a fully isolated LAN using VMs and private zones under our control. We emulated clients, resolvers, and multiple authoritative servers; no experiment generated traffic toward the public Internet. Firewalls blocked any egress beyond the testbed. These experiments did not interact with third-party infrastructure and could not affect external users.

End-to-end BADDNS experiments. End-to-end experiments in Section 5.1 were conducted by triggering fragmentation behaviors against authoritative nameservers on the real network to evaluate whether the attack could effectively impact resolvers and thus demonstrate feasibility. In all such tests, the target resolver was under our control, and any poisoned cache was confined to this isolated resolver instance. No malicious content was served to external users on the public Internet.

Internet-scale measurements. We assessed real-world exposure with conservative probing in our controlled domains.

Public DNS services (Section 6.1). For each service IP, we issued at most 15 queries per IP (nonexistent long name and www lookups), spacing requests by more than 1s. We only observed whether lookups for our own domain were redirected and cached; we did not attempt to poison or degrade service.

Open resolvers (Section 6.2). We first identified DNSSEC-validating open resolvers by querying domains with valid/invalid RRSIGs three times per IP, and then retained only resolvers returning NOERROR for the former and SERVFAIL for the latter. For the poisoning feasibility check, we sent exactly one long-name-trigger query

and one validation query per IP, spaced by more than 0.5s. This single-shot design minimizes load for the target resolvers.

Authoritative nameservers (Section 6.3). For each DNSSEC-signed nameserver discovered from Tranco Top-1M domains, we sent two long-name queries and a single ICMP Packet-Too-Big message to test fragmentation behavior. ICMP packets used a source address under our administrative control and complied with local egress policies. We measured only fragmentation and split-point placement; we did not alter target zone data, nor did we attempt cache poisoning against live resolvers.

Across all Internet measurements, probe volumes were small (a handful of packets per target) and spaced to avoid burstiness. Scanner IPs had PTR records pointing to a project page describing the study and providing opt-out contact. We honored opt-out requests; none were received.

Data handling and privacy. We did not collect or process user-level identifiers. Measurements concern resolver/authoritative behavior only. Results are reported in aggregate; we do not publish enumerations of vulnerable IPs or domains to prevent misuse. Raw logs were stored on closed research machines and retained only as needed to reproduce aggregate statistics.

Responsible disclosure. As discussed in Section 7.2, we responsibly disclosed BADDNS to the affected vendors, including DNS software, public DNS providers, and hosting service providers. Also, we proposed strict semantics for BAD caches and DNSSEC deployment guidelines in Section 7.1 for countering BADDNS.

Furthermore, we carefully consider the effects on stakeholders if this work is published, as follows.

Resolvers. Our work demonstrates that resolvers can still face cache poisoning risks even when they support DNSSEC. Publication can motivate resolver vendors to counter the BADDNS attack, and end users can receive more stable and secure service. PowerDNS has already released a version that includes the patch.

Nameservers. Publication helps authoritative server operators verify whether their servers are vulnerable to the BADDNS attack. We provide a checking tool during disclosure to vendors, including Cloudflare, which has confirmed the issue.

DNS Community. The community is already aware that DNS packet fragmentation is harmful and has proposed mitigations to prevent fragmented responses, thereby making them less exploitable by attackers. However, our work shows that a non-negligible set of authoritative servers still allows response fragmentation, making BADDNS feasible in the real world. Publishing this work can draw renewed attention to this problem.

B Open Science

To enable independent evaluation of our findings while adhering to ethical norms, we reviewed all datasets, binaries, and code used in this study and classified what we can safely release. Below we enumerate the artifacts¹, their availability, and the rationale for any redactions.

PoC. The PoC reproduces BADDNS in a closed lab. It includes:

(i) Configuration for a DNSSEC-signed hosting zone for DNS hosting, under both configurations (Name Error and Wildcard);

(ii) A fragmentation simulator that avoids IP spoofing, including the simulation of two modifications (Insert NS_IP and Insert Glue).

The bundle was inspired by Dart [68] for rapid DNS vulnerability testbed construction.

Withheld for ethics and safety. We do not release artifacts that could enable real-world exploitation:

(i) any code that performs IP source-address spoofing and raw fragment forgery;

(ii) any code that scans vulnerable nameservers in the wild.

C Fragmentation-based Cache Poisoning

Fragmentation-based poisoning takes a different route from forgery-based attacks: it leaves all entropy-bearing bytes in the legitimate first fragment and alters only the second fragment, which contains the trailing DNS payload but no UDP header, no source port, and no question section. Prior work shows that if a resolver reassembles the legitimate first fragment with an attacker-crafted second fragment, the result can be a syntactically valid packet whose DNS body has been changed, bypassing defenses aimed at whole-packet forgeries.

PTB-driven PMTU control. Our goal is to make the authoritative nameserver’s reply split into at least two IPv4 fragments on the path to the resolver. We induce fragmentation by sending ICMP Packet Too Big (IPv4 Type 3, Code 4) messages toward the authoritative server for the hosting zone (e.g., ns1.vitm-ns.com), advertising a smaller next-hop MTU. When the server honors these messages, its oversized UDP responses will be fragmented accordingly on the subsequent path to the resolver.

Pre-injecting the forged last fragment. IPv4 reassembly associates fragments by the source and destination IP addresses, the protocol (UDP), and the IP identifier (IPID). A forged last fragment must match these values to be merged with the legitimate first fragment at the resolver. The forged packet is crafted as the last fragment (the “More Fragments” flag is cleared). Its fragment offset is chosen so that the forged payload begins exactly where the authoritative first fragment is expected to end; in other words, it aligns with the start of the Authority/Additional tail we intend to alter. Because this fragment contains only trailing DNS bytes, the attacker has full control over its content. The forged last fragment is sent slightly before the resolver receives the authoritative reply.

IPID prediction. We rely on existing, well-known IPID inference techniques from prior work [69]. Recent Internet measurements [24] further show that a non-trivial share of authoritative nameservers (e.g., 5.1% among Tranco-top domains) maintain globally incrementing IPID counters, which lowers the bar for matching. In addition, common Linux-based operating systems maintain a small cache of response fragments (e.g., 64 entries), each retained for on the order of tens of seconds (about 30s) [9]; this allows multiple parallel guesses to be staged within a single reassembly window.

Meet the UDP checksum. After reassembly, the full UDP datagram must pass checksum verification. Because the attacker controls all bytes in the second fragment, a small “compensation” adjustment can be inserted into a writable field so that the overall one’s complement sum is correct. Practically, we embed this correction into a value that is both under our control and semantically harmless to adjust (for example, the TTL of an injected A record, or an EDNS option value). This technique is standard in prior works and does not require altering any entropy in the first fragment.

¹Artifacts available at Zenodo: <https://zenodo.org/records/20772172>